Cómo llevo un prototipo de IA a producción y lo mantengo

Revisado el 18 de octubre de 2026

TRES VOCES, SIEMPRE SEPARADAS

● LO QUE HICE	◆ LO QUE EL CAMPO HACE	▲ MI POSICIÓN
Sale solo del corpus de este sitio.	Sale de una fuente con título, editor, fecha y enlace.	Es mi criterio, no un hecho.

Lo que no lleva rótulo es mi método: mi posición sobre cómo se hace.

BLOQUE 1

La pregunta

¿Cómo pasas del prototipo a producción, y cómo lo mantienes sano, barato y seguro?

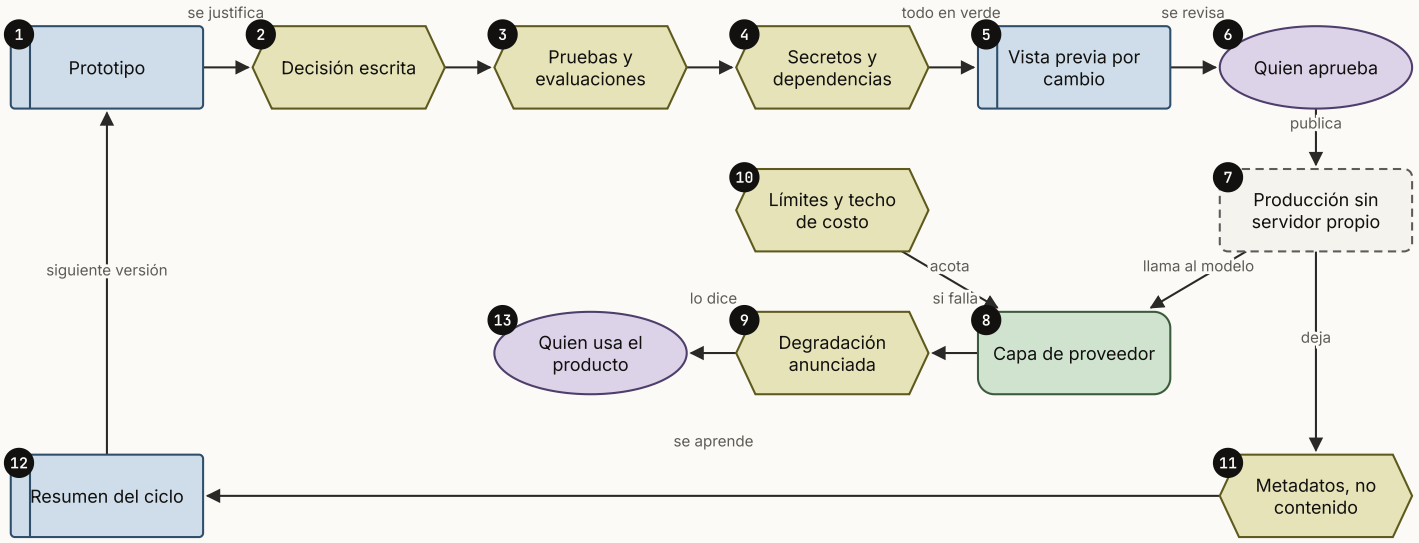
LA RESPUESTA EN TRES LÍNEAS

1. Un prototipo pasa a producción cuando supera condiciones escritas que ya se vieron fallar. Los controles que corren antes de que publique son las pruebas unitarias, de integración y de extremo a extremo, la accesibilidad, el rendimiento y el bloqueo de secretos, y si hay un modelo, preguntas de prueba que corren en cada cambio como cualquier otra prueba.
2. Antes de la IA va el código: el modelo entra solo donde una regla no alcanza, detrás de una capa que permite cambiar de proveedor y con una salida más simple, anunciada, para cuando falla.
3. Publicar es el comienzo de la validación: lo mantengo con metadatos y no con el contenido de quien lo usa, con límites y un techo de costo escritos, y con un resumen de cada ciclo que convierte la falla en regla.

BLOQUE 2

El mapa

Del prototipo a producción, y de producción a la siguiente versión



LEYENDA DE FORMAS Y COLORES

Actor — quien hace el trabajo

Dato — lo que se guarda y se lee

Control — una regla que se comprueba

Persona — una persona decide

Externo — lo que viene de afuera

LECTURA DEL MAPA, NODO POR NODO

Un prototipo empieza con una decisión escrita: si necesita un modelo, por qué una regla no basta. Después cruza los controles de publicación —pruebas en tres niveles, accesibilidad, rendimiento y, si hay modelo, preguntas de prueba—, el bloqueo de secretos y la revisión de dependencias. Con todo en verde, el cambio se despliega en una vista previa propia, una persona la revisa y decide publicarlo. En producción, sin servidor propio, el producto llama al modelo a través de una capa que cambia de proveedor por configuración; los límites y el techo de costo acotan esas llamadas, y si el proveedor falla, el producto ofrece una función más simple y se lo dice a quien lo usa. La operación deja metadatos —no contenido— y cada ciclo cierra con un resumen y una decisión registrada, de donde parte la versión siguiente.

Nodo 1 **Prototipo** (Dato) — Algo que ya funciona en una máquina: todavía no es un producto.

Nodo 2 **Decisión escrita** (Control) — Código primero: un modelo entra solo con una decisión de arquitectura que diga qué no resuelve una regla.

Nodo 3 **Pruebas y evaluaciones** (Control) — Unitarias, de integración y de extremo a extremo, accesibilidad, rendimiento y, si hay modelo, preguntas de prueba: todas en cada cambio.

Nodo 4 **Secretos y dependencias** (Control) — Un secreto se bloquea antes de entrar al historial; una dependencia vulnerable frena el cambio.

Nodo 5 **Vista previa por cambio** (Dato) — Cada cambio se despliega aparte y se mira antes de llegar a producción.

Nodo 6 **Quien aprueba** (Persona) — Una persona revisa la vista previa y, con cada control requerido en verde por sí mismo, decide publicar.

Nodo 7 **Producción sin servidor propio** (Externo) — Páginas estáticas, funciones en el borde y una base gestionada con seguridad por filas: nada que operar a mano.

Nodo 8 **Capa de proveedor** (Actor) — El único módulo que conoce proveedores concretos: una variable elige cuál, y fallas seguidas abren el circuito.

Nodo 9 **Degradación anunciada** (Control) — Sin proveedor, el producto ofrece una función más simple que conserva lo esencial, y lo dice.

Nodo 10 **Límites y techo de costo** (Control) — Frecuencia, tamaño, tokens, tiempo e historial escritos como números, y un presupuesto mensual que no decide solo la factura.

Nodo 11 **Metadatos, no contenido** (Control) — Proveedor, modelo, milisegundos, tokens y errores: lo necesario para saber qué falló, dónde y cuánto costó.

Nodo 12 **Resumen del ciclo** (Dato) — Cada ciclo cierra con lo que falló y por qué; una falla que se repite se vuelve regla con su prueba.

Nodo 13 **Quien usa el producto** (Persona) — Recibe la respuesta completa o la función simple, y en los dos casos sabe cuál recibió.

CONEXIONES

de Prototipo a Decisión escrita (se justifica)

de Decisión escrita a Pruebas y evaluaciones

de Pruebas y evaluaciones a Secretos y dependencias

de Secretos y dependencias a Vista previa por cambio (todo en verde)

de Vista previa por cambio a Quien aprueba (se revisa)

de Quien aprueba a Producción sin servidor propio (publica)

de Producción sin servidor propio a Capa de proveedor (llama al modelo)

de Límites y techo de costo a Capa de proveedor (acota)

de Capa de proveedor a Degradación anunciada (si falla)

de Degradación anunciada a Quien usa el producto (lo dice)

de Producción sin servidor propio a Metadatos, no contenido (deja)

de Metadatos, no contenido a Resumen del ciclo (se aprende)

de Resumen del ciclo a Prototipo (siguiente versión)

BLOQUE 3

Las fases

1

FASE 1

Decidir si hace falta un modelo

ENTRADAS

- El problema y el proceso donde vive.
- Lo que una regla, una búsqueda o un algoritmo ya resuelven.

ACTIVIDADES

- Escribir qué parte exige interpretar, generar, recuperar por contexto o coordinar herramientas.
- Resolver primero con código lo que el código resuelve.
- Dejar la decisión registrada, con lo que el modelo no debe hacer.

ARTEFACTOS

- Una decisión de arquitectura escrita.

QUIÉN DECIDE

Quien responde por el producto.

CONTROL QUE CIERRA LA FASE

Sin decisión escrita, la función generativa no se activa.

> • LO QUE HICE
>
> En mis apps activar una función generativa exige una decisión de arquitectura escrita: Innmobiliaria tiene cero modelos por regla de producto, y donde sí hay uno es opcional y con un fallback determinista.

2

FASE 2

Convertir el prototipo en algo verificable

ENTRADAS

- El prototipo.
- Las condiciones de publicación, escritas.

ACTIVIDADES

- Pruebas unitarias, de integración y de extremo a extremo; accesibilidad y presupuesto de rendimiento.
- Si hay modelo: preguntas de prueba y casos donde debe abstenerse, declarar que no sabe o rechazar.
- Ver fallar cada control nuevo frente a lo que pretende atrapar, antes de darlo por bueno.

ARTEFACTOS

- La suite en la integración continua.
- La demostración en rojo de cada control nuevo.

QUIÉN DECIDE

Quien construye.

CONTROL QUE CIERRA LA FASE

Un control nuevo entra solo después de verse fallar.

◆ LO QUE EL CAMPO HACE

Microsoft recomienda incorporar pruebas cualitativas en los pipelines de publicación, porque la IA no es determinista, y que el monitoreo alimente las puertas de calidad en todo el ciclo.[8]

3

FASE 3

Proteger lo que se publica

ENTRADAS

- El repositorio, que puede ser público.
- Las credenciales del proveedor y de la base de datos.

ACTIVIDADES

- Bloquear los secretos antes del commit y probar ese bloqueo con una carnada que sí lo dispara.
- Dejar las claves solo en las variables de entorno del despliegue, nunca en el código que llega al navegador.
- Probar cada credencial antes de construir contra ella.
- Auditar las dependencias y actualizarlas en lotes pequeños, verificados uno a uno.

ARTEFACTOS

- El bloqueo de secretos y su carnada verificada.
- La configuración del entorno, fuera del repositorio.

QUIÉN DECIDE

Quien construye.

CONTROL QUE CIERRA LA FASE

Un commit con un secreto no llega a existir.

◆ LO QUE EL CAMPO HACE

GitHub activó por defecto la protección contra secretos en los push a repositorios públicos: si detecta uno, bloquea el push hasta que se quite o se decida pasarlo por alto. [9]

4

FASE 4

Antes de publicar: corren los controles y una persona decide

ENTRADAS

- La vista previa del cambio.
- Las conclusiones de cada control requerido.

ACTIVIDADES

- Revisar la vista previa como la verá quien la usa.
- Comprobar que cada control requerido corrió y terminó en verde por sí mismo: uno saltado no es verde.
- Publicar desde la rama principal, con la versión anterior a la mano para volver.

ARTEFACTOS

- El cambio en producción.
- Una forma de volver a la versión anterior.

QUIÉN DECIDE

Quien es dueño del producto.

CONTROL QUE CIERRA LA FASE

Ningún control requerido saltado ni en rojo.

> • LO QUE HICE
>
> A cada control le hago tres preguntas antes de darlo por bueno: ¿lo viste fallar?, ¿lo viste correr? —un job saltado no es verde— y ¿puede fallar siquiera?

5

FASE 5

Operar: medir, acotar y degradar

ENTRADAS

- El uso real.
- El presupuesto escrito.

ACTIVIDADES

- Registrar metadatos estructurados y reportar los errores, sin el contenido de quien usa el producto.
- Poner límites de frecuencia, longitud, salida, tiempo e historial.
- Abrir el circuito tras fallas seguidas y pasar a la función simple, diciéndolo.
- Tener un interruptor que apague la función generativa sin tumbar el resto.

ARTEFACTOS

- Registros estructurados y alertas de errores.
- El modo degradado, probado.

QUIÉN DECIDE

Quien paga y responde por la operación.

CONTROL QUE CIERRA LA FASE

Con el proveedor caído, el producto sigue cumpliendo lo esencial y lo dice.

> • LO QUE HICE
>
> En el chat de este sitio, tras tres fallas consecutivas el circuito se abre y el navegador busca en el mismo índice, diciendo que no es una respuesta del modelo.

6

FASE 6

Aprender y empezar la siguiente versión

ENTRADAS

- Las fallas, las mediciones y lo que encontró quien usa el producto.

ACTIVIDADES

- Escribir el resumen del ciclo: qué se hizo, qué falló y qué quedó pendiente.
- Convertir el incidente en una regla con su prueba, no en una anécdota.

ARTEFACTOS

- El resumen y la decisión registrada.
- La regla nueva y su demostración en rojo.

QUIÉN DECIDE

Quien es dueño del método.

CONTROL QUE CIERRA LA FASE

Sin resumen escrito, el ciclo no cierra.

> • LO QUE HICE
>
> En el sprint 3 de este sitio una credencial del proveedor devolvió un 401 en la integración; desde entonces toda credencial pasa una prueba de humo antes de construir contra ella, y la integración continua la repite en cada corrida.

BLOQUE 4

Las decisiones de diseño

Código primero o un modelo

CUÁNDO SÍ

Un modelo, cuando el problema exige interpretar, generar, recuperar por contexto o coordinar herramientas con flexibilidad, y una regla no alcanza.

CUÁNDO NO

Cuando una regla, una búsqueda estructurada o un algoritmo cumplen el propósito: son más previsibles, más baratos y más fáciles de probar.

POR QUÉ

• LO QUE HICE

Es la regla 13 de mi pipeline: antes de usar IA generativa demuestro qué característica del problema la exige, y activarla pide una decisión de arquitectura escrita.

▲ MI POSICIÓN

La pregunta no es si puedo meter IA, sino si la mejora justifica lo que trae: dependencias, costo, variabilidad y una evaluación que alguien tiene que sostener.

Un proveedor fijo o una capa que lo cambia

CUÁNDO SÍ

Una capa conmutable, cuando el producto debe sobrevivir a que un modelo se retire, una cuota cambie o un proveedor se caiga.

CUÁNDO NO

Un proveedor fijo, cuando la función depende de algo que solo uno ofrece y esa dependencia se acepta por escrito.

POR QUÉ

• LO QUE HICE

En el chat una variable de entorno elige entre cinco proveedores adaptados, y una capa de adaptación es el único módulo que los conoce: cambia quien redacta, no el índice ni los hechos.

▲ MI POSICIÓN

Cambiar de proveedor es fácil cuando el conocimiento no vive en él; lo que hay que probar es que el nuevo conserve las citas, los rechazos y el tamaño de la respuesta.

Qué se evalúa en cada cambio y qué se revisa a mano

CUÁNDO SÍ

En cada cambio, lo que se mide de forma barata y determinista: que la recuperación traiga su fuente, que los destinos existan, que las preguntas ajenas se corten.

CUÁNDO NO

Un juez con modelo en cada commit, cuando cuesta más de lo que protege: ahí va por lotes, antes de publicar.

POR QUÉ

◆ LO QUE EL CAMPO HACE

Microsoft documenta una acción de GitHub que evalúa agentes dentro de la integración continua, con intervalos de confianza y pruebas de significancia para saber si un cambio mejoró o solo varió, y aconseja no correrla en cada commit para contener el costo.[3]

● LO QUE HICE

En este sitio la recuperación se evalúa en cada cambio con preguntas propias y de afuera; la respuesta que intenta meter algo no recuperado la caza hoy la lectura, no un gate.

Contenedores propios o despliegue sin servidor

CUÁNDO SÍ

Sin servidor propio, cuando el producto es estático o de funciones cortas y nadie debería tener que operar una máquina: el borde y el navegador alcanzan.

CUÁNDO NO

Cuando la carga pide procesos largos, estado en memoria o un control del entorno que la plataforma no da: ahí un contenedor paga lo que cuesta operarlo.

POR QUÉ

● LO QUE HICE

Mis despliegues son sin servidor propio —Vercel, Cloudflare Workers, el navegador del usuario— y por eso nunca han necesitado un contenedor.

◆ LO QUE EL CAMPO HACE

Para cargas de IA, Microsoft recomienda empezar por plataforma como servicio y opciones sin servidor para simplificar la adopción y la operación, y dejar el autoalojamiento para necesidades avanzadas.[8]

Qué se guarda para observar

CUÁNDO SÍ

Metadatos —proveedor, modelo, milisegundos, tokens, errores— para saber qué falló, dónde y cuánto costó.

CUÁNDO NO

El contenido de quien usa el producto, salvo que el producto lo necesite, se diga antes de la primera palabra y tenga su decisión registrada.

POR QUÉ

> • LO QUE HICE
>
> En mis apps lo que se observa viaja como metadatos por Pino y Sentry; el chat de este sitio sí guarda pregunta y respuesta desde el 21 de septiembre de 2026, y lo advierte antes de que el visitante escriba.

> ◆ LO QUE EL CAMPO HACE
>
> Microsoft Foundry (antes Azure AI Foundry) traza cada llamada al modelo y a las herramientas sobre el estándar OpenTelemetry, y su monitoreo de producción sigue consumo de tokens, latencia, tasa de errores y calidad, con alertas cuando una salida no pasa sus umbrales.[2]

BLOQUE 5

Lo innovador en 2026

◆ LO QUE EL CAMPO HACE

Microsoft Foundry, una plataforma que junta evaluar, observar y poner barreras

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

Microsoft Foundry reúne agentes, modelos y herramientas en un solo recurso, con trazas, monitoreo y evaluaciones incorporados; sus barreras de seguridad pueden revisar la entrada, la salida y, para agentes y en vista previa, la llamada a una herramienta y su respuesta.[1][12]

> • LO QUE HICE
>
> Mi experiencia con Microsoft Foundry, completa en el bloque 9: lo conecté al chat como uno de sus proveedores, y estudio su temario en la ruta del AI-103, que está en curso.

◆ LO QUE EL CAMPO HACE

Las evaluaciones como una prueba más del pipeline

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

La observabilidad deja de empezar en producción: Foundry integra puertas de calidad automáticas en la integración continua, aplica evaluadores de calidad, de RAG y de seguridad a un conjunto de consultas, y después evalúa una muestra del tráfico real.[2][3]

> ● LO QUE HICE
>
> En este sitio las preguntas de prueba ya corren en cada cambio como cualquier otra prueba; para la respuesta del modelo no tengo un juez automático.

◆ LO QUE EL CAMPO HACE

Atacarse a uno mismo antes de publicar

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

El agente de red teaming de Foundry prueba la aplicación con estrategias de ataque de PyRIT y mide la tasa de ataques exitosos; Microsoft advierte que el juez es generativo, que puede dar falsos positivos y que los resultados se revisan antes de actuar.[4]

> ▲ MI POSICIÓN
>
> Lo correría antes de abrir al público una función generativa nueva, y leería los ataques que pasaron antes de tocar un solo filtro.

◆ LO QUE EL CAMPO HACE

Mandar cada pedido al modelo que alcanza

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

El enrutador de modelos de Foundry analiza cada pedido y lo manda a un modelo pequeño cuando basta y a uno de frontera cuando lo exige; dice qué modelo respondió, y Microsoft aclara que no quita la necesidad de evaluar.[5]

> ● LO QUE HICE
>
> El ahorro de mi chat ocurre antes del modelo: las preguntas fuera de alcance no consumen tokens y solo viajan los fragmentos relevantes, no el sitio entero.

◆ LO QUE EL CAMPO HACE

Reutilizar el principio del prompt

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

Si los primeros 1.024 tokens de un pedido son idénticos a uno anterior, el servicio reutiliza el cómputo y cobra con descuento esa lectura; por eso lo estable va al principio y lo que cambia, al final.[6]

▲ MI POSICIÓN

Es una palanca cuando las instrucciones son largas y se repiten idénticas; antes de contar con ella mediría cuánto del pedido es de verdad estable.

◆ LO QUE EL CAMPO HACE

El consumo sin límite, tratado como riesgo de seguridad

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

El Top 10 de OWASP para aplicaciones con LLM de 2025 trata el consumo sin control como riesgo —denegación de servicio, pérdidas económicas— y pide validar el tamaño de las entradas, limitar la frecuencia por origen, poner tiempos máximos y vigilar el uso de recursos.[10]

● LO QUE HICE

Mi chat tiene esos límites escritos como números y no como intenciones: 800 caracteres por pregunta, 700 tokens de salida, 30 segundos y 12 mensajes de historial.

◆ LO QUE EL CAMPO HACE

La cadena de suministro también es la de la IA

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

OWASP extiende la cadena de suministro a modelos, datos y plataformas, y pide revisar a los proveedores, mantener un inventario de componentes al día —una lista de materiales de software — y una política de parches para lo vulnerable o desactualizado.[11]

● LO QUE HICE

Anonimizador Velo audita sus dependencias en la integración continua contra tres familias vetadas de SDK de IA, y una dependencia transitiva cuenta igual que una directa.

◆ LO QUE EL CAMPO HACE

Operar IA ya es una certificación

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

La guía de estudio de Microsoft para su certificación de operaciones de aprendizaje automático y de IA generativa pide versionar los prompts en Git, montar evaluaciones automáticas con métricas de calidad y de seguridad, y seguir el costo en tokens junto a la latencia y el rendimiento.[7]

> • LO QUE HICE
>
> Sigo la ruta del AI-300 desde julio de 2026; está en curso y no la presento como obtenida.

BLOQUE 6

Modos de falla y cómo se ven venir

QUÉ SE ROMPE	CÓMO SE VE VENIR	CONTROL QUE LA ATRAPA
Un control que siempre da verde	Nadie lo vio fallar; un job saltado aparece entre los requeridos sin alarma.	La demostración en rojo viaja en el mismo commit que el control, y cada verificación requerida tiene su propia conclusión.
Un secreto llega al repositorio	Una clave en un commit, en un registro o en el código que llega al navegador.	Barrido de secretos antes del commit, probado con una carnada que sí lo dispara; claves solo en el entorno del despliegue.
Una credencial inválida se descubre al final	El primer uso real devuelve un 401.	Prueba de humo de cada credencial antes de construir contra ella, repetida en la integración continua.

QUÉ SE ROMPE	CÓMO SE VE VENIR	CONTROL QUE LA ATRAPA
El proveedor se cae, se queda sin cuota o retira el modelo	Fallas seguidas, tiempos vencidos, un modelo que deja de existir.	Capa de proveedor conmutable, cortacircuitos y una función degradada que se anuncia como tal.
El costo crece sin que nadie lo decida	Más tokens por respuesta, pedidos ajenos que llegan al modelo, picos de un solo origen.	Un filtro antes del modelo, límites de frecuencia y tamaño, y un techo de costo escrito.
La calidad se degrada en silencio después de publicar	Reintentos que se acumulan con una integración, respuestas que dejan de citar, latencia que sube.	Telemetría por sesión con metadatos y las preguntas de prueba corriendo contra cada cambio.
Una dependencia vulnerable o desactualizada	Un aviso de seguridad, o un paquete que baja de versión sin que nadie lo pida.	Auditoría de dependencias en la integración continua y actualizaciones en lotes pequeños, cada uno verificado.
La telemetría se lleva datos de quien usa el producto	Contenido de un usuario en un registro o en un error reportado.	Registro estructurado solo con metadatos; lo que sí se guarda se declara antes y tiene su decisión escrita.

BLOQUE 7

Métricas y controles

QUÉ SE MIDE	UMBRAL	QUIÉN APRUEBA EL PASO A PRODUCCIÓN
Controles requeridos con conclusión propia en verde	Todos; uno saltado cuenta como no verde.	Quien publica, leyendo cada conclusión.
Controles nuevos vistos fallando	Todos, antes de entrar.	Quien construye, con la demostración registrada.
Preguntas de prueba con su fuente entre las que entran	Un piso escrito que no baja de un cambio al siguiente.	Quien es dueño del contenido.
Secretos en el historial	Cero: uno solo detiene el commit.	El bloqueo de secretos, sin excepciones.
Costo mensual frente al techo escrito	Por debajo del techo; si se acerca, se recorta contexto y extensión antes de cambiar de modelo.	Quien paga la operación.
Latencia y errores del proveedor	Dentro del tiempo máximo por pedido; fallas seguidas abren el circuito.	Quien responde por el producto.
Función esencial con el proveedor caído	Disponible y rotulada como degradada.	Quien responde por el producto.

QUÉ SE MIDE	UMBRAL	QUIÉN APRUEBA EL PASO A PRODUCCIÓN
Contenido de usuarios en la telemetría	Cero.	Quien responde por los datos.

Los umbrales son ejemplos del método, no cifras de un proyecto mío.

BLOQUE 8

Mi evidencia

• LO QUE HICE USO REAL

La cadena de publicación que comparten mis apps

Mis aplicaciones hermanas y este sitio comparten la misma cadena: Git y GitHub, integración continua con pruebas unitarias, de integración y de extremo a extremo, accesibilidad con axe, presupuesto de rendimiento con Lighthouse y despliegue con una vista previa por cambio; ninguna rama llega a producción con uno de esos trabajos en rojo.

Ver dónde →

• LO QUE HICE USO REAL

Este sitio: la integración continua contra una base real

El trabajo de integración levanta un Supabase real, le aplica las migraciones y prueba contra Postgres; un permiso de la base se da por verificado cuando una prueba intenta saltárselo y no puede.

Ver dónde →

• LO QUE HICE USO REAL

Un control se demuestra fallando

Cada control nuevo trae en el mismo commit su demostración en rojo. La regla nació el 15 de julio de 2026, cuando el barrido de secretos pasó en silencio dos veces con una carnada floja: desde entonces la carnada canónica está verificada.

• LO QUE HICE USO REAL

La capa de proveedor y la degradación, probadas en una falla real

Una variable de entorno elige el proveedor del chat. En el sprint 3 una credencial devolvió un 401 en la integración, el circuito se abrió y la búsqueda local respondió sobre el mismo índice, diciendo que no era una respuesta del modelo.

Ver dónde →

• LO QUE HICE USO REAL

Un techo de costo y el costo medido

El chat opera con un techo de US$20 al mes y un costo real observado de US$0, porque el uso cabe en la cuota gratuita del proveedor inicial y las consultas ajenas se resuelven sin invocarlo: es una medición, no una promesa de gratuidad.

• LO QUE HICE USO REAL

Observar agentes en producción

En Vesting diseñé la plataforma que observaba hasta 23 agentes a la vez en producción: los eventos viajaban a Microsoft Fabric y de ahí a Power BI, y la telemetría por sesión mostró a un agente que acumulaba reintentos antes de que un usuario lo reportara.

Ver dónde →

• LO QUE HICE USO REAL

Datos con seguridad por filas y acceso solo por funciones

La base de este sitio tiene la seguridad por filas encendida y sin políticas: el rol anónimo solo ejecuta funciones con privilegios del definidor, y la llave de servicio no viaja al navegador ni vive en el repositorio.

• LO QUE HICE USO REAL

Modelos en producción, sostenidos

La mitad de MLOps que consiste en llevar modelos a producción y sostenerlos la he hecho: modelos predictivos en Banco Pichincha y en TransMilenio, el monitoreo de agentes en Vesting y aplicaciones públicas con integración continua y despliegue automático, operadas por mí.

BLOQUE 9

Lo que haría distinto y lo que no he hecho

• LO QUE HICE

Contenedores en producción con Docker y Kubernetes no están en mi experiencia: Kubernetes lo conozco como estudio, no como operación, y la mitad de MLOps que empaqueta y orquesta con contenedores no la he hecho.

• LO QUE HICE

Con Microsoft Foundry mi experiencia es el adaptador de proveedor de este chat, un ajuste de un modelo que hice ahí y el estudio del AI-103, que está en curso; no describo ningún despliegue propio en Azure.

• LO QUE HICE

El AI-300 está en curso: es la certificación de operar modelos y agentes, y no la presento como obtenida.

● LO QUE HICE

La respuesta del modelo todavía se revisa leyendo: la que intenta introducir información no recuperada la caza la lectura, no un gate.

● LO QUE HICE

Las fases de lanzamiento y de operación de mi fábrica de apps siguen sin validarse en una corrida real: son método escrito, no experiencia.

▲ MI POSICIÓN

Lo que haría distinto hoy: la evaluación con un juez calibrado como un trabajo más de la integración continua desde el primer sprint con IA, y un red teaming automatizado antes de abrir una función generativa al público.

▲ MI POSICIÓN

Y no empezaría por la plataforma: empezaría por las condiciones de publicación, escritas y vistas fallar; la plataforma se elige después, por lo que esas condiciones necesitan.

BLOQUE 10

Fuentes y fecha de revisión

Revisado el 18 de octubre de 2026

[1] What is Microsoft Foundry?
Microsoft Learn · 13 de agosto de 2026
https://learn.microsoft.com/en-us/azure/foundry/what-is-foundry

[2] Observability in Generative AI
Microsoft Learn · 15 de octubre de 2026
https://learn.microsoft.com/en-us/azure/foundry/concepts/observability

[3] How to run an evaluation in GitHub Action
Microsoft Learn · 30 de abril de 2026
https://learn.microsoft.com/en-us/azure/foundry/how-to/evaluation-github-action

[4] AI Red Teaming Agent
Microsoft Learn · 19 de agosto de 2026
https://learn.microsoft.com/en-us/azure/foundry/concepts/ai-red-teaming-agent

[5] How model router works in Microsoft Foundry
Microsoft Learn · 9 de septiembre de 2026
https://learn.microsoft.com/en-us/azure/foundry/openai/concepts/model-router-how-it-works

[6] Prompt caching with Azure OpenAI in Microsoft Foundry Models
Microsoft Learn · 11 de agosto de 2026
https://learn.microsoft.com/en-us/azure/foundry/openai/how-to/prompt-caching

[7] Study guide for Exam AI-300: Operationalizing Machine Learning and Generative AI Solutions
Microsoft Learn · 18 de febrero de 2026
https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/ai-300

[8] MLOps and GenAIOps for AI Workloads on Azure (Azure Well-Architected Framework)
Microsoft Learn · 1 de noviembre de 2024
https://learn.microsoft.com/en-us/azure/well-architected/ai/mlops-genaiops

[9] Keeping secrets out of public repositories
GitHub · 29 de febrero de 2024
https://github.blog/news-insights/product-news/keeping-secrets-out-of-public-repositories/

[10] LLM10:2025 Unbounded Consumption (OWASP Top 10 for LLM Applications 2025)
OWASP GenAI Security Project · 17 de noviembre de 2024
https://genai.owasp.org/llmrisk/llm102025-unbounded-consumption/

[11] LLM03:2025 Supply Chain (OWASP Top 10 for LLM Applications 2025)
OWASP GenAI Security Project · 17 de noviembre de 2024
https://genai.owasp.org/llmrisk/llm032025-supply-chain/

[12] Guardrails and controls overview in Microsoft Foundry
Microsoft Learn · 31 de julio de 2026
https://learn.microsoft.com/en-us/azure/foundry/guardrails/guardrails-overview